



Universidad
Indoamérica

UNITÉ 6 · FONDEMENTS DES SYSTÈMES D'EXPLOITATION

Signaux et mémoire partagée

Universidad Tecnológica Indoamérica

Laboratoire TD5

Signaux: notifications asynchrones à un processus

Le noyau

SIGSEGV (accès invalide), SIGBUS, SIGPIPE (tube sans lecteur), SIGCHLD (un enfant s'est terminé), SIGALRM

Le terminal

Ctrl-C → SIGINT, Ctrl-\ → SIGQUIT, Ctrl-Z → SIGTSTP, fermeture → SIGHUP

Un autre processus

`kill(pid, sig)` et la commande `kill` : SIGTERM, SIGKILL, SIGUSR1...

Ils changent l'état du processus: ils le terminent, l'arrêtent, le relancent... ou exécutent l'une de ses fonctions.

Envoi asynchrone, signaux en attente et moment de leur capture

- L'envoi peut avoir lieu à tout moment, que la cible soit en cours d'exécution ou non
- **En attente** : reçu mais pas encore délivré (bloqué, ou le noyau ne l'a pas encore vérifié)
- Il est délivré au **retour en mode utilisateur** : après un appel système ou un changement de contexte

Précision sur l'original (« il n'existe pas de file de signaux ») : c'est vrai pour les signaux *standard* — un signal en attente est un bit, les répétitions se perdent —, mais les signaux **temps réel** (`SIGRTMIN` ... `SIGRTMAX`) sont bien mis en file et peuvent porter une valeur (`sigqueue`).

kill et les signaux les plus courants

C

```
int kill(pid_t pid, int sig);  
/* pid > 0: that process · 0: my process group  
   < -1: group -pid · -1: every process I may signal  
   errors: EINVAL, EPERM, ESRCH */
```

Correction de l'original (diapositive 97): il disait que SIGHUP, SIGINT, SIGTSTP... sont « non masquables et non bloquables ». Seuls **SIGKILL** et **SIGSTOP** ne peuvent être ni capturés, ni ignorés, ni bloqués; tous les autres le peuvent.

SIGNAL	PAR DÉFAUT
SIGHUP · SIGINT · SIGTERM	terminer
SIGQUIT · SIGSEGV · SIGBUS	terminer + core
SIGKILL	terminer (toujours)
SIGPIPE · SIGALRM · SIGUSR1 · SIGUSR2	terminer
SIGSTOP · SIGTSTP	arrêter
SIGCONT	continuer
SIGCHLD	ignorer

En attente, bloqués et délivrés

signal	n	en attente	bloqué	action	par défaut
SIGINT (Ctrl-C)	2	0	—	SIG_DFL	terminer
SIGQUIT (Ctrl-\)	3	0	—	SIG_DFL	terminer + core
SIGKILL	9	0	impossible	SIG_DFL	terminer
SIGUSR1	10	1	oui	gestionnaire	terminer
SIGSEGV	11	0	—	SIG_DFL	terminer + core
SIGPIPE	13	0	—	SIG_DFL	terminer
SIGALRM	14	0	—	SIG_DFL	terminer
SIGCHLD	17	0	—	SIG_DFL	ignorer
SIGCONT	18	0	—	SIG_DFL	continuer
SIGSTOP	19	0	impossible	SIG_DFL	arrêter
SIGTSTP (Ctrl-Z)	20	0	—	SIG_DFL	arrêter
SIGRTMIN	34	0	—	SIG_DFL	terminer

processus

endormi dans pause / sigsuspend

ce qui s'est passé

- sigprocmask(SIG_BLOCK, {SIGUSR1}) ; while (!flag)...
- kill(pid, SIGUSR1) : le noyau marque SIGUSR1 comme en attente
- il arrive au pire moment, mais reste EN ATTENTE (bloqué)
- sigsuspend(masque sans SIGUSR1) : débloque et attend en UNE seule opération

Gestionnaires: de `signal` à `sigaction`

C

```

sighandler_t signal(int sig, sighandler_t f); /* obsolete; SIG_ERR on error */
int sigaction(int sig, const struct sigaction *act,
              struct sigaction *old);
struct sigaction {
    void (*sa_handler)(int); /* or SIG_DFL, SIG_IGN */
    void (*sa_sigaction)(int, siginfo_t *, void *);
    sigset_t sa_mask; /* blocked during the handler */
    int sa_flags; /* SA_RESTART SA_SIGINFO SA_NOCLDSTOP... */
};

```

- Pendant le gestionnaire, le même signal reste bloqué
- Au retour, le programme reprend où il en était. `sa_flags` : `SA_RESTART` relance l'appel interrompu (sinon `EINTR`, unité 2); `SA_SIGINFO` utilise `sa_sigaction` ; `SA_NOCLDSTOP` : `SIGCHLD` seulement quand un enfant se termine, pas quand il s'arrête ou reprend
- À l'intérieur, uniquement des fonctions **async-signal-safe** (`write` oui, `printf` et `malloc` non) et des variables `volatile sig_atomic_t`
- `signal()` a une sémantique différente selon les systèmes: utilisez `sigaction`

Attendre et programmer: `pause`, `alarm`, `sleep`

- `pause()` : dort jusqu'à ce qu'un signal le termine ou exécute un gestionnaire; renvoie toujours -1
- `alarm(s)` : SIGALRM dans `s` secondes (par défaut, termine le processus)
- `sleep(s)` : suspend pendant `s` secondes

Mise à jour (diapositive 101) : l'original disait que `sleep` est implémenté avec `pause` + `alarm` et qu'il ne faut pas les mélanger. Avec glibc/Linux, `sleep` utilise `clock_nanosleep` et ne touche pas à l'alarme. Aujourd'hui on préfère `nanosleep` et les temporisateurs `timer_create` ou `timerfd`.

Les exemples de l'original... et leur bogue caché

c

```
/* slide 102: mysleep */
signal(SIGALRM, do_nothing);
alarm(s);
pause();      /* what if SIGALRM arrives BEFORE pause? */

/* slide 103: ping-pong, parent and child after fork() */
kill(getppid(), SIGUSR1);
pause();      /* same race: the answer may come first */
```

- Si le signal arrive **entre** la préparation et `pause()`, le gestionnaire s'exécute trop tôt et `pause` attend pour toujours (diapositive 105)
- Un bogue difficile à détecter et à reproduire: il dépend de l'ordonnancement (comme la course de `mypipe`, unité 5)
- La conclusion de l'original (diapositive 104): **n'utilisez pas** `signal()` / `pause()` pour synchroniser

Bloquer, vérifier et `sigsuspend`

C tds/td5/mysignal.c

```
/* Block SIGUSR1 outside sigsuspend: a signal arriving
   and the wait stays pending instead of being lost (th
sigemptyset(&bblock);
sigaddset(&bblock, SIGUSR1);
exit_if(sigprocmask(SIG_BLOCK, &bblock, &wait_mask) == -1);
sigdelset(&wait_mask, SIGUSR1);

while (received < max)
    sigsuspend(&wait_mask);          /* atomically: unbl
```

`sigprocmask` bloque; `sigsuspend` **débloque et dort en une seule opération atomique.**

Opérations sur les ensembles: `sigemptyset`,
`sigfillset`, `sigaddset`, `sigdelset`,
`sigismember`.

Sortie réelle

```
$ ./mysignal 2 & P=$!; sleep 0.3; kill -USR1 $P; sleep
0.2; kill -STOP $P; kill -USR1 $P; sleep 0.2; kill -CONT
$P; sleep 0.2; kill -USR1 $P; wait $P; echo "exit status:
$? (128 + 10 = killed by SIGUSR1)"
pid 357338: waiting for SIGUSR1 (2 times)
handler: received signal 10
handler: received signal 10
default action restored: the next SIGUSR1 terminates me
bash: line 1: 357338 User defined signal 1
./mysignal 2
exit status: 138 (128 + 10 = killed by SIGUSR1)
```

Signal + mémoire: l'émetteur et le récepteur

Sortie réelle

```
$ ./receiver & sleep 0.3; echo 'hello from the sender' | ./sender; wait
receiver 357391: waiting for SIGUSR1
sender: the receiver is 357391
received: hello from the sender
```

- Le récepteur écrit son PID au début d'un fichier projeté et attend SIGUSR1 avec `sigsuspend`
- L'émetteur lit le PID, écrit le message dans le même fichier projeté et envoie le signal: le signal **prévient**, la mémoire **transporte**

Aujourd'hui aussi: `signalfd` (lire les signaux comme des données depuis un descripteur),
`pidfd_send_signal` (envoyer un signal sans course liée à la réutilisation des PID).

Partager de la mémoire entre processus

- La mémoire virtuelle isole: chaque processus a ses pages (unité 4)
- Partager = faire pointer des pages virtuelles de **deux processus** vers le **même cadre physique**
- Il faut un **point de rendez-vous**: un fichier ou un nom

mmap + fichier

le fichier est le rendez-vous et stocke les données (persiste)

shm_open +
mmap

POSIX, un nom dans /dev/shm (en RAM), recommandé aujourd'hui

ftok +
shmget +
shmat

System V, clé numérique; le contenu du fichier n'est pas utilisé

Projeter un fichier en mémoire: `mmap`

C

```
void *mmap(void *addr, size_t len, int prot, int flags,  
           int fd, off_t offset); /* MAP_FAILED on error */  
int munmap(void *addr, size_t len);  
int msync(void *addr, size_t len, int flags);
```

- `prot` : `PROT_NONE`, `PROT_READ`, `PROT_WRITE`, `PROT_EXEC`
- `flags` : `MAP_SHARED`, `MAP_PRIVATE` (copie sur écriture), `MAP_ANONYMOUS`
- Erreurs: `EACCES` (pas un fichier ordinaire, ou le mode du descripteur ne permet pas ce qui est demandé: `O_WRONLY` et `O_APPEND` ne fonctionnent pas), `ENOMEM`

Correction de l'original (diapositive 118):

`len` n'a pas besoin d'être un multiple de la taille de page (il est arrondi au-dessus); c'est `offset` qui doit l'être. Et accéder au-delà de la fin du fichier ne donne pas `SIGSEGV` mais **`SIGBUS`**: l'animation suivante le montre avec la sortie réelle.

Partager de la mémoire par un fichier

C tds/td5/sender.c

```
int fd = open(FILENAME, O_RDWR);
exit_if(fd == -1, "open " FILENAME);
char *shared = mmap(NULL, SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
exit_if(shared == MAP_FAILED, "mmap");
exit_if(close(fd) == -1, "close");

...

shared[off] = '\\0';

exit_if(msync(shared, SIZE, MS_SYNC) == -1, "msync"); /* optional: also push to the file */
exit_if(kill(pid, SIGUSR1) == -1, "kill");
exit_if(munmap(shared, SIZE) == -1, "munmap");
```

- `O_RDWR` : `PROT_WRITE` + `MAP_SHARED` exige un descripteur en lecture-écriture
- Après `mmap`, on peut fermer le descripteur: la projection reste active
- Écrire `shared[i]`, c'est écrire le fichier: le récepteur, qui projette le même fichier, le voit
- `munmap` supprime la projection; `msync` force la copie sur le disque

Les limites de `mmap` : `SIGBUS` et `PROT_*`

page 0 (0 ... 4095)

zone lisible/inscriptible au-delà du fichier: se lit comme 0, n'est pas enregistrée

fichier: 128 o

octets écrits avec `map[i] = 'x'`

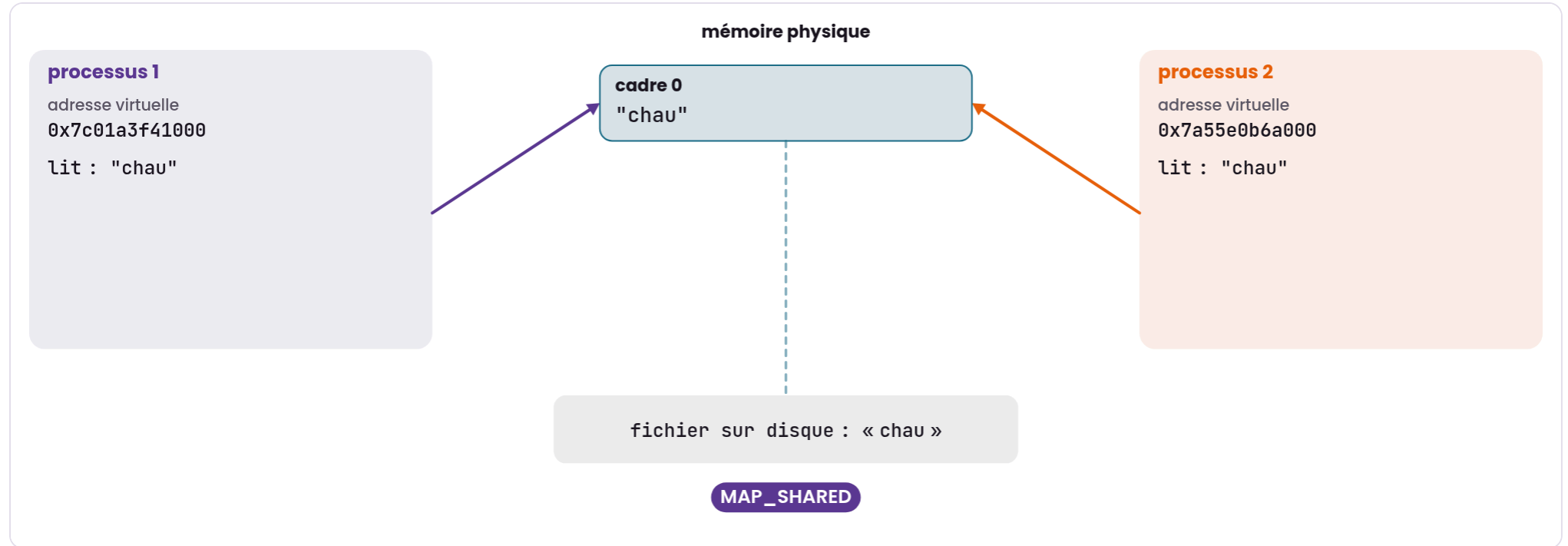
page 1 (4096 ... 8191)

pas de fichier derrière → `SIGBUS`

`SIGBUS @ 4096`

taille du fichier ensuite: 128 o

Deux processus, un cadre physique



Mémoire partagée System V: `ftok` → `shmget` → `shmat`

C tds/td5/shm_counter.c

```
key_t key = ftok(KEYFILE, KEYID);
exit_if(key == -1, "ftok");
int id = shmget(key, SIZE, IPC_CREAT | 0644);
exit_if(id == -1, "shmget");

if (argc > 1 && strcmp(argv[1], "--remove") == 0) {
    exit_if(shmctl(id, IPC_RMID, NULL) == -1, "shmctl I");
    printf("segment %d removed\n", id);
    return EXIT_SUCCESS;
}

int *counter = shmat(id, NULL, 0);
exit_if(counter == (void *)-1, "shmat");
counter[0]++; /* a fresh
printf("segment %d (key 0x%08x): counter = %d\n", id, (
exit_if(shmdt(counter) == -1, "shmdt");
```

Sortie réelle

```
$ ./shm_counter; ./shm_counter; ./shm_counter;
./shm_counter --remove
segment 65593 (key 0x4605476d): counter = 1
segment 65593 (key 0x4605476d): counter = 2
segment 65593 (key 0x4605476d): counter = 3
segment 65593 removed
```

- Le segment survit aux processus jusqu'à `shmctl(IPC_RMID)` ; `ipcs -m` les liste
- Chaque processus le voit à une autre adresse: **n'y stockez jamais de pointeurs**
- `IPC_PRIVATE` : segment sans clé · `SHM_RDONLY` : lecture seule · `shmctl` : `IPC_STAT` lit, `IPC_SET` change les permissions, `IPC_RMID` détruit

Partager de la mémoire exige de synchroniser

Sans opération atomique

```
( slot = seg[0] + 1; seg[0] = slot; ):
```

Sortie réelle

```
$ for i in 1 2 3 4 5; do ./shm_race 5 | cut -d: -f1; done
19 processes registered (expected 32)
28 processes registered (expected 32)
25 processes registered (expected 32)
23 processes registered (expected 32)
27 processes registered (expected 32)
```

Avec `__atomic_add_fetch` :

Sortie réelle

```
$ ./shm_fork
16 processes registered (expected 16): 357564 357566
357567 357572 357568 357571 357569 357565 357574 357573
357570 357576 357578 357575 357577 357579
```

C'est le sujet de l'unité 7: conditions de course, mutex, sémaphores.

TD5 corrigé + défi IA

- `mysignal` avec `sigsuspend` ; l'émetteur et le récepteur
- `mmap_limits` et les cinq protections
- `shm_counter`, `shm_fork` et la course `shm_race`

Bash

```
make -C tds && tds/tests/run.sh td5
```



Universidad
Indoamérica

À RETENIR

Trois questions

1. Pourquoi trois SIGUSR1 bloqués n'exécutent-ils le gestionnaire qu'une seule fois?
2. Quel problème `sigsuspend` résout-il que `pause` ne résout pas?
3. Pourquoi ne stocke-t-on pas de pointeurs en mémoire partagée?